Esercizi Laboratorio Sistemi operativi (**clicca per indice**)

Laboratorio 01

Esercizio 1: Processi vs Thread

Obiettivo: Sfruttare una differenza tra processi e thread nella **gestione della memoria** per rendere ancora meno reattiva la versione multi-processo, a vantaggio di quella multi-thread, e verificare l'aumento dello speedup.

Come funzionano i programmi di misura:
- Sorgenti: **reactivity_processes.c** e **reactivity-threads.c**.
- Per N volte si crea un processo (**fork** + **wait**) o un thread (**pthread_create** + **pthread_join**) che esegue **do_work()** e terminated.
- Si misura il tempo medio di lancio + terminazione con la libreria **performance**.
- Lo speedup è calcolato come:

$$\text{speedup} = \frac{\text{tempo reactivity process}}{\text{tempo thread process}}$$

- Nei sorgenti originali **do_work()** è vuota, quindi la differenza tra processi e thread è piccola.

Differenza da sfruttare:
- Con **fork()** il processo figlio riceve una copia dell'intera memoria del padre. Su Linux la copia avviene in **copy-on-write**: una pagina viene duplicata solo quando il figlio la scrive;
- I thread di un processo condividono la memoria, quindi non hanno questo costo.

Attenzione (**copy-on-write**):
- Con **fork()** il figlio lavora su una copia dello spazio di indirizzamento del padre.
- I thread di uno stesso processo condividono lo spazio di indirizzamento.

Idea:
Far scrivere a **do_work()** su un grande buffer globale, una volta per ogni pagina:
- **processi**: ogni figlio paga il costo di ogni pagina che tocca (page fault + pagina nuova o copiata);
- **thread**: le pagine sono condivise, il costo si paga una volta sola.

do_work() va modificata allo stesso modo nei due sorgenti.

Soluzione variante 1: buffer allocato nel main con calloc

Codice: Definizioni e **do_work()**, identiche in reactivity-processes.c e reactivity-thread.c:

```
#define ITEMS (1 << 24)   // 2^24 int = 64 MB
#define STEP  1024        // 1024 int = 4 KB
int* global_buff = NULL;

void do_work() {
    int j;
    for (j = 0; j < ITEMS; j += STEP) {
        global_buff[j] = j;   // una scrittura per ogni pagina da 4 KB
    }
}
```

Nel main, prima delle misure:

```
global_buff = (int*)calloc(ITEMS, sizeof(int));
if (global_buff == NULL) { ... }
... // misure invariate
```

Spiegazione del codice:
- **ITEMS** è la dimensione del buffer (64 MB di **int**), abbastanza grande da rendere visibile il costo del **fork()**.
- **STEP = 1024** int = 4 KB = una pagina: il ciclo avanza di una pagina alla volta e scrive un valore per pagina, quindi tocca tutte le 16.384 pagine con il minio lavoro. Il lavoro in sé è trascurabile: conta l'effetto sulla memoria.
- Il buffer è allocato e azzerato nel **main** prima delle misure, quindi la **calloc** non entra nei tempi.

Cosa succede nei due casi:
- **Processi**: il padre non scrive mai nelle pagelle, quindi il kernel le mappa come pagine azzerate, allocate solo alla prima scrittura. Ogni figlio, a ogni **fork()**, alloca 16.384 pagine nuove (page fault + pagina nuova).
- **Thread**: le pagine sono allocate dal primo thread e poi condivise da tutti. Non c'è page fault né copia a ogni lancio; il ciclo di scritture costa comunque qualcosa, ma molto meno.

Alternativa alla variante 1: malloc + memset

Cambia un solo dettaglio rispetto alla variante 1: come viene preparato il buffer. Invece della **calloc**, il padre alloca con **malloc** e azzera con **memset,** sempre nel **main** e prima delle misure (**do_work()** resta identica).

```
#include <string.h>

global_buff = (int*)malloc(ITEMS * sizeof(int));
if (global_buff == NULL) { ... }
memset(global_buff, 0, ITEMS * sizeof(int));   // il padre scrive in tutte le pagine
... // misure invariate
```

Così il padre tocca tutte le pagine, che quindi esistono già in memoria fisica al momento del **fork()**:
- **Processi**: i figli le copiano alla scrittura (vero copy-on-write: page fault + copia della pagina). In più **fork()** deve copiare le tabelle delle pagine del padre, e il costo cresce con la memoria mappata;
- **Thread**: nessun costo di allocazione, le pagine sono già condivise.

Soluzione, variante 2: modificare solo do_work()

Vincolo: non toccare il **main** (sono ammesse variabili globali). Si usa un array globale non inizializzato:

```
#define ITEMS (1 << 24)   // 2^24 int = 64 MB
#define STEP  1024        // 1024 int = 4 KB
int global_buff[ITEMS];   // non inizializzato: niente calloc, nessuna modifica al main

void do_work() {
    int j;
    for (j = 0; j < ITEMS; j += STEP)
        global_buff[j] = j;   // una scrittura per pagina
}
```

- Il kernel mappa **global_buff[ITEMS]** come pagine azzerate, allocate solo alla prima scrittura: stesso comportamento della **calloc** da 64 MB.
- **Processi**: il padre non lo tocca mai, quindi ogni figlio alloca 16.384 pagine nuove a ogni **fork()**.
- **Thread**: pagine allocate dal primo thread, poi condivise da tutti.
- Il **main** è identico a quello del codice di partenza.

Quanto conta il valore di STEP (N=300):
- **STEP = 1**: anche i thread fanno 16M scritture, il cui costo domina in entrambi i casi. quindi lo speedup è minimo.
- **STEP < 1024**: le pagine da duplicare sono le stesse ma il numero di scritture per pagina si riduce, quindi il costo delle copie conta sempre di più.
- **STEP = 1024** (4 KB / **sizeof(int)**): una sola scrittura per pagina, quindi il rapporto tra costo delle copie e costo delle scritture è massimo.
- **STEP > 1024**: si saltano pagine, quindi meno copie, e i costi fissi di **fork()**/**exit** pesano di più, quindi lo speedup cala.

Compilazione ed esecuzione:
Con N basso, perché ogni **fork()** è ora lento:

```
gcc -o reactivity-processes reactivity-processes.c performance.c -lpthread -lrt -lm
gcc -o reactivity-threads reactivity-threads.c performance.c -lpthread -lrt -lm
./reactivity-processes <N>
./reactivity-threads <N>
```

Per la variante 2 la compilazione e l'esecuzione sono invariate, con i sorgenti modificati.

Risultati: Misurati con l'implementazione **malloc** + **memset** (256 MB, un byte per pagina), sulla macchina di prova con 2 core. Sulla tua macchina i valori assoluti saranno diversi.

	Processi	Thread	Speedup
Versione originale (N = 1000)	197 µs	67 µs	≈ 2,9
Versione modificata (N = 200)	258.619 µs	2.933 µs	≈ 88

Conclusione: Facendo lavorare i processi figli su una grande quantità di memoria, il costo del copy-on-write (o dell'allocazione della pagine) fa crescere molto il tempo dei processi. I thread, che condividono lo spazio di indirizzamento, lo pagano massimo si ottiene con una scrittura per pagine (**STEP = 1024**).

Esercizio 2: Accesso concorrente a variabili condivise

Obiettivo: Capire cosa succede quando più thread scrivono in concorrenza su una variabile condivisa, e risolvere il problema senza meccanismi di sincronizzazione, lavorando sulle strutture dati.

- Sorgente di partenze: **concurrent_threads.c**.
- Il programma lancia **N thread** in parallelo; ciascuno aggiunge **M volte un valore V** a una variabile condivisa, inizializzata a 0.
- Al termine il **main** verifica che gli incrementi equivalgono complessivamente a $N \cdot M \cdot V$.

Il problema (**add** perse): Se tutti i thread incrementano la stessa variabile, l'operazione $+=$ non è atomica: ogni thread legge il valore, lo somma e lo riscrive, e se due thread leggono lo stesso valore prima che uno dei due riscriva, un incremento va perso. E' una **race condition**.

Esempio. (N = 1000, M = 10000, V = 1)

```
$ ./concurrent_threads 1000 10000 1
The value of the shared variable is 8674815. It should have been 10000000
Number of lost adds: 1325185
```

Il valore atteso è 10.000.000, ma ne risultano circa 1,3 milioni in meno.

Soluzione 1: una cella per thread

L'idea è far si che **nessun thread scriva sulla stessa locazione di un altro**:

- ogni thread incrementa una locazione di memoria diversa (una cella di un array);
- alla fine il **main** somma tutti i valori.

Sorgente: **sol-concurrent_threads.c**

```
unsigned long int* shared_array;    // una cella per thread

void* thread_work(void *arg) {
    int thread_idx = *((int*)arg);  // legge il proprio indice dall'argomento
    int i;
    for (i = 0; i < m; i++)
        shared_array[thread_idx] += v;  // scrive SOLO nella propria cella
    return NULL;
}

// main
shared_array = calloc(n, sizeof(unsigned long int));   // n celle azzerate
int* thread_ids = (int*)malloc(n * sizeof(int));       // un indice per thread
for (i = 0; i < n; i++) {
    thread_ids[i] = i;
    pthread_create(&threads[i], NULL, thread_work, &thread_ids[i]);  // indirizzo diverso per ogni thread
}
for (i = 0; i < n; i++) {
    pthread_join(threads[i], NULL);          // aspetta il thread i
    computed_value += shared_array[i];       // legge la cella i solo DOPO la join
}
```

Spiegazione passo per passo:

1. **calloc** crea un array con **una cella per thread**, già azzerata.
2. Ogni thread riceve un **indirizzo diverso** (**&thread_ids[i]**), da cui legge il proprio indice, e scrive **solo nella propria cella**: nessuna scrittura concorrente.

3. Il **main** legge la cella **i** solo **dopo la pthread_join** del thread **i**, quindi il thread ha già finito di scrivere.
4. Il **main** somma tutte le celle in **computed_value**.

Compilazione ed esecuzione:

```
gcc -o sol-concurrent_threads sol-concurrent_threads.c -lpthread
./sol-concurrent_threads <N> <M> <V>
```

Non dovrebbero risultare **add** perse.

Esempio. (N = 1000, M = 10000, V = 1)

```
$ ./sol-concurrent_threads 1000 10000 1
The value computed on the array is 10000000. It should have been 10000000
```

Cosa succede se al posto di &thread_ids[i] si usa 6i?
Il codice sembra funzionare, ma è sbagliato:
- non c'è alcuna garanzia su **quando** verrà eseguita l'istruzione **int thread_idx = *((int*)arg);**;
- nel frattempo il valore nella locazione puntata da **arg** può cambiare: è il contatore **i**, che il **main** continua a incrementare (e riusa nel ciclo di join);
- il risultato è che più thread possono avere la stessa identità;
- si propone il problema dell'accesso concorrente.

Soluzione alternativa: valore restituito con pthread_exit
- Ogni thread accumula su una **variabile propria** e restituisce il valore tramite **pthread_exit**.
- Il **main** raccoglie i valori tramite **pthread_join** e li somma,
- Non servono già ne l'array condiviso ne gli indici dei thread.

Sorgente: **sol2-concurrent_threads.c**

```
void* thread_work(void *arg) {
    unsigned long* portion = calloc(1, sizeof(unsigned long));  // risultato nell'heap
    int i;
    for (i = 0; i < m; i++)
        *portion += v;              // accumula sulla variabile propria
    pthread_exit(portion);          // restituisce il puntatore al risultato
}

// main
for (i = 0; i < n; i++)
    pthread_create(&threads[i], NULL, thread_work, NULL);   // nessun argomento

unsigned long computed_value = 0;
unsigned long* thread_result;
for (i = 0; i < n; i++) {
    pthread_join(threads[i], (void**) &thread_result);  // riceve il puntatore passato a
pthread_exit
    computed_value += *thread_result;
    free(thread_result);                        // il main libera la memoria
}
```

Note:
- Il **risultato va nell'heap**: una variabile locale vive nello stack del thread, che non esiste più dopo la sua terminazione.

- **Nessun argomento**: ogni thread non ha bisogno di sapere chi è.
- **pthread_join** restituisce il puntatore passato a **pthread_exit**; il **main** lo libera con **free**.

Compilazione ed esecuzione:

```
gcc -o sol2-concurrent_threads sol2-concurrent_threads.c -lpthread
./sol2-concurrent_threads <N> <M> <V>
```

Conclusione: Per evitare le add perse senza meccanismi di sincronizzazione basta che **nessuna locazione sia scritta da più di un thread**: o dando a ogni thread una cella dell'array condiviso (soluzione 1), oppure facendo accumulare ogni thread su una variabile propria e raccogliendo i risultato con **pthread_join** (soluzione alternativa). In entrambi i casi il **main** somma i valori solo dopo la **join**, e il risultato è esattamente $N \cdot M \cdot V$.

INDICE

Laboratorio 01 **1**

Esercizio 1: Processi vs Thread 1

Esercizio 2: Accesso concorrente a variabili condivise 4